Energy-Efficient Virtual Machines Placement

Albert P. M. De La Fuente Vigliotti[1], **Daniel Macêdo Batista**[1]

[1]Department of Computer Science – University of São Paulo (USP)
Rua do Matão, 1010 – Cidade Universitária – 05508-090 – São Paulo – SP – Brazil

{albert, batista}@ime.usp.br

Abstract. *Energy efficiency on computer systems is a topic that is gaining a lot of interest. Even more in the cloud computing era, where data centers consumption corresponds to near 1.5% of total world wide power consumption. In this paper we present two novel approaches for virtual machines (VMs) placement consolidation. The two approaches aim to maximize the placed VMs on a host and therefore minimize the number of hosts used on a cloud computing environment. The first proposed approach is based on the Knapsack problem and the second one is based on an Evolutionary Computation heuristic. Both strategies have shown consumed energy reduction starting from 40.33% and up to 92.21% compared to a strategy that does not consider energy efficiency.*

1. Introduction

In the age of climate change and global warming, energy efficiency has become one of the most important design requirements for computing systems such as data centers and cloud systems. Data centers consume enormous amounts of electrical power, which leads to significant emissions of carbon dioxide into the environment. For example, the current IT infrastructure is responsible for about 2% of total world wide power consumption and CO_2 footprints. This corresponds to the typical electricity consumption of 120 million households per year. This consumption problem is rapidly growing [Beloglazov et al. 2010, Fettweis and Zimmermann 2008, Rich Brown 2007].

The performance per watt ratio of computers has been constantly increasing every year. If this trend continues, the cost of the energy consumed by a server during its lifetime will exceed the hardware acquisition cost. The problem is even worse for clusters, data centers and large-scale compute infrastructures. An energy consumption growth of 16-20% per year can be observed in the last years, corresponding to a doubling every 4-5 years [Beloglazov et al. 2010].

As reported by members of the Open Compute Project [The Open Compute Project Foundation 2013], Facebook's Oregon data center is one of the most energy efficient data center in the world. The data center had a power usage effectiveness (PUE) of 1.09, which means that more than 91% of energy is consumed by the computing resources. Now it is important to focus on resources management efficiency.

In this paper we propose two energy efficient strategies for virtual machines placement on homogeneous cloud environments. The proposal considers several resources like CPU, memory, network and I/O utilization. The first approach is based on a Knapsack algorithm and the second one is based on an Evolutionary Computation (EC) algorithm. Both approaches are compared with a strategy that doesn't consider energy efficiency.

Experiments showed significant energy savings starting from 40.33% and up to 92.21% on simulations performed with real workloads. Besides, a framework in Python has been developed along with the algorithms to facilitate the simulations.

The reminder of this paper is structured as follows: a more detailed overview on the concepts used can be found on the rest of this section. Afterward we review the related work on Section 2. We introduce the problem on Section 3 and the algorithms on Section 4. The framework developed during this work is discussed on Section 5. The performance evaluation is presented on Section 6. Conclusions and future directions can be found on Section 7.

1.1. Virtualization

Virtualization is a technology that allows to create logical computing units called Virtual Machines (VMs) that can accommodate an individual OS, allowing a physical server behavior. These VMs are able to run applications transparently isolated from the physical hosts where the VMs run.

When talking about virtualization there is an important choice to make: isolation of the guest and host vs. resources sharing. *Hypervisor-based virtualization* (VMware [VMware, Inc. 2013], Xen [Linux Foundation Collaborative Project 2013] and KVM [Redhat Emerging Technologies 2013]) prioritizes isolation of resources, libraries and OS from each VM from the host, over sharing, by running a full OS both on the guest and on the host with the Consequent overhead. As technology evolved and many hardware virtualization extensions like Intel-VT and AMD-V got better over time, the performance got better as well. Nevertheless there is still a performance gap, specially regarding I/O operations.

With recent developments of *container-based virtualization* CGroups (Control Groups) [Jackson and Lameter 2013], Linux Containers (LXC - *"chroot on steroids"*) [LXC Development Group 2013], OpenVZ [Community project, supported by Parallels, Inc. 2013] and Linux-VServer [Linux-VServer Development Group 2013] there was a step forward achieving almost native performance by using the same host OS within the guest VM by sharing the host libraries. Other typical scenarios include separating applications to improve security, hardware independence and migration for load balancing of containers in a cluster.

Recent studies have shown how container based virtualization could be used even in High Performance Computing (HPC) environments [Xavier et al. 2013], that is still not even an option with hypervisor-based virtualization technology. The two approaches share advantages, but some features are unique to each virtualization platform (e.g., full network virtualization, migration, etc.) therefore some users still prefer hypervisors instead of containers. In terms of energy savings, the use of containers is better because it does not have the overhead of having two operating systems. In this work we consider that virtualization is managed by containers and that it is possible to retrieve each resource usage percentage at any time.

1.2. Cloud Computing

There are many definitions for *Cloud computing*, often related to cloud and web services for marketing reasons, but cloud computing is more than that. A complete definition has

been made by the *National Institute of Standards and Technology (NIST)*:

"Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction..." [Mell and Grance 2011]

Cloud computing has many benefits, among them we can highlight: (a) *costs savings* – organizations no longer have to invest money in their own IT infrastructure, as services are available on demand paid as they are used, so there are decreasing investments and running costs; (b) *efficiency* – cloud providers can operate better, faster and in a more cost-efficiently way than own IT infrastructure; (c) *improve availability* – it is common to find load balancing, redundant connections and state of the art infrastructure setup to meet business needs. It also improves (d) *scalability* – services can be scaled for business needs as resources can be allocated dynamically [Brian et al. 2008].

2. Related Work

There are some studies regarding optimization of energy consumption on hypervisor-based virtualized cloud computing environments, The work [Beloglazov and Buyya 2013] proposes an approach for solving the problem of host overload detection. They maximize the mean inter-migration time under the specified QoS goal based on a Markov chain model. The algorithm handles unknown non-stationary workloads using a multisize sliding window workload estimation technique.

OpenStack Neat [Beloglazov and Buyya 2012] is an open source software framework for distributed dynamic VM consolidation in cloud data centers based on the OpenStack platform. The framework acts independently of the OpenStack platform and applies VM consolidation processes by invoking public Application Programming Interfaces (APIs) of OpenStack.

CloudSim [Rajkumar Buyya 2013] is an extensible cloud simulation framework developed in Java that enables seamless modeling, simulation, and experimentation of cloud computing infrastructures. It supports modeling and simulation of energy-aware computational resources.

The VMs consolidation approaches cited until now are limited to a single resource utilization analysis (e.g., CPU), nevertheless we believe that other resources indicators like: network, I/O and memory could be used to improve the decision making process. These resource indicators can be retrieved by using container based virtualization which motivated this work.

Regarding containers-based virtualization there is the work [Xavier et al. 2013], but we noticed that they focused only on evaluation and comparing performance with hypervisor-based virtualization of high performance computing environments (HPC) and not regular cloud systems. They also presented that it seems to be a consensus that today's hypervisor-based virtualization systems perform well for CPU intensive applications, but exhibit a high overhead when handling I/O intensive applications, especially the network intensive ones. They do not study energy aspects as we propose.

As presented in [Soltesz et al. 2007], container-based systems provide up to 2x the performance of hypervisor-based systems for server-type workloads and scale further

while preserving performance. In general, we notice a lack of studies on the container-based virtualization energy efficiency area, and that LXC support for OpenStack is very limited. This might be due to the fact that containers technology has been integrated on the Linux Kernel recently and by consequence is a topic full of opportunities.

3. Energy Efficient VM Allocation

Allocation of VMs can be divided in two problems, the first one is the placement of a new VM on a host while the second one is the optimization of the current VMs allocation. In this paper we focus on the first problem.

Most of the studies we found observes the CPU utilization and makes their decision based on it. This makes sense since there is a relationship between the total power consumption by a server and its CPU utilization as observed by [Fan et al. 2007]. Basically their model proposes that power consumption by a server grows linearly with the growth of the CPU utilization. It is interesting to notice that in data centers, as observed by [Barroso and Holzle 2007], the mean value of the CPU utilization is 36.44% with 95% confidence interval.

CPU utilization based models are able to provide an accurate prediction for CPU-intensive applications, however they tend to be inaccurate for other types of prediction like I/O and memory intensive applications as concluded by [Dhiman et al. 2010]. For this reason our proposal is to include other resource usage indicators to try to increase the accuracy of the predictions.

We notice that the available cloud computing simulators like CloudSim [Rajkumar Buyya 2013] make their decision only based on CPU observation, hence we developed a VM placement simulation framework in Python called pyCloudSim [1][Vigliotti and Batista 2014] which considers other resources like network, memory and I/O usage. It would be easy to customize the code to include new resource indicators on future works.

Energy efficient VM allocation takes advantage of the fact that idle hosts can be suspended to low-power modes to reduce the overall energy consumption. Later the hosts can be reactivated remotely over the network by using Wake-on-LAN when an increase of resources use is detected. It is important to carefully evaluate where consolidation of the VMs is going to take place, we do not want to migrate VMs to an overloaded host. It is very important to always keep in mind the Quality of Service (QoS) so we do not fall into Service Level Agreements (SLA) violations. This is more like the second type of problem (optimization of the current VMs allocation).

The process to allocate virtual machines considered in our work is based on the work proposed in [Beloglazov and Buyya 2012]. This process consists in performing periodical dynamic consolidation of VMs by packing them on fewer physical machines as possible to conserve energy in virtualized data centers. The reason besides this idea is that computers are more efficient when they are operating near 100% of their capacities. This approach can be divided into the following sub-problems.

- **Information retrieval**: as stated before, most current VMs consolidation approaches are limited to CPU utilization analysis. We can include other resources

[1] `https://github.com/vonpupp/pyCloudSim/tree/v0.1`

like network, I/O and memory.

- **Underload detection**: it's important to detect hosts that can be considered underloaded because they will be operating with a bad energy efficiency. In this case all the VMs from this host should be migrated to another host (if possible) and later the "old host"should be switched to a low-power mode.
- **Overload detection**: similar to the previous step, to ensure Quality of Service (QoS) requirements, if a host is considered overloaded, some VMs should be migrated to another host, either active or in low-power mode hosts. The last ones need to be reactivated.
- **VMs selection**: select which VMs to migrate from an overloaded host.
- **VMs migration**: migrate the selected VMs to other hosts (active or suspended).

4. Proposed Approaches

This section presents the two proposed approaches to allocate VMs considering the energy consumption of physical machines modeled with CPU and other resources: network interfaces, memory and I/O. We propose two different approaches to solve the problem, the first one is based on a Knapsack strategy while the second one is based on an Evolutionary Computation (EC) strategy.

We can define the problem as follows. Given sets of:

- Physical hosts $H = \{H_1, \ldots, H_n\}$ of size n
- Available resources $\vec{C}_p = (C_{p,1}, \ldots C_{p,d})$ of size d (dimensions)
- Virtual Machines $V = \{V_1, \ldots, V_m\}$ of size m
- Used resources $R = \{R_1, \ldots, R_d\}$ of size d (dimensions)

Each physical host H_i has a d-dimensional available resource capacity as a vector $\vec{C}_i = (C_{i,1}, \ldots C_{i,d})$, i.e CPU, memory, network, etc. All the virtual machines resources usage $vm_j \in V$ are represented by a d-dimensional resource demand vector $\vec{r}_j = (r_{j,1}, \ldots, r_{j,d}) \in [0, 1]^d$.

The objective of our approaches is to minimize the physical hosts used, which can also be seen as maximize the number of VMs per host constrained to the resources usage. Moreover, the approaches are not naive in terms of SLA so we also aim to avoid a high rate of blocked virtual machine allocations, which can happen if there are no available physical machine to attend the demand of new virtual machine requests.

4.1. The Knapsack Approach

In this first approach we model the VMs placement problem as a Multidimensional Bin Packing (MDBP) approach. In this approach the following decision variables need to be found:

Physical host allocation variable $h_p = \begin{cases} 1 & \text{if the physical host } p \text{ is used by some VM} \\ 0 & \text{otherwise} \end{cases}$

Virtual machine allocation variable $x_{i,p} = \begin{cases} 1 & \text{if the virtual machine } i \text{ is assigned to the host } p \\ 0 & \text{otherwise} \end{cases}$

The goal is to minimize the number of physical hosts,

$$\text{Minimize} \sum_{p=1}^{n} h_p \tag{1}$$

Subject to the following constraints:

$$\sum_{i=1}^{m} r_{i,k} \times x_{i,p} \leq C_{p,k} \times h_p, \forall p \in \{1, \ldots, n\}, \forall k \in R \quad (2)$$

$$\sum_{i=1}^{n} x_{i,p} = 1, \forall i \in \{1, \ldots, m\} \quad (3)$$

Where constraint (2) ensures that the capacity of each physical host is not exceeded and constraint for each resource (3) guarantees that each virtual machine is assigned to exactly one physical host.

MDBP is a variation of the Multiple-Choice Multidimensional Knapsack Problem (MMKP) which is a Non-deterministic Polynomial-time hard (NP-Hard) problem. This type of problem needs to be solved by heuristic algorithms. We first intended to solve MDBP problems limited to three dimensions (i.e. CPU, network and I/O), but we noticed that the algorithm did not scale well due to its complex nature, therefore we reduced the problem to solve the standard knapsack problem n-times (one per host). This strategy showed to be far more scalable than the first one, since we could run experiments with more than 280 VMs and 100 hosts in just a few seconds, while the MDBP did not scale for more than 10 hosts without taking a considerable amount of time. We named this approach as: *Iterated-KSP*, which is how we will refer to it for the rest of this work.

A Python library called `OpenOpt` [Kroshko 2013] has been used for the development of the Iterated-KSP strategy.

In Algorithm 1, a list of constraints is built (line 2) for each d-dimension. The summation of each resource used by the VMs within a host must be less than 100%. The constraints also include assigning a weight on each VM which will be the criteria to be maximized by the algorithm. In the simulation we used equal weights for all the VMs. Nevertheless, according to different strategies, this could be changed. The placement decisions for that host is performed using the `KSP` function (line 3), i.e. maximize the number of VMs to be placed on that host. As the host is initially without any VM, it is filled with its full computing capacity for each d-dimension without exceeding any dimension.

Algorithm 1 Iterated-KSP Strategy

Input: Number of physical hosts: $\mathscr{P}$
Input: Number of virtual machines: $\mathscr{V}$

1: **function** ITERATED-KSP-SCENARIO(...)
2: constraints ← build constraints for every d-dimension to be ≤ 100
3: placement ← maximize VMs using `KSP` with $\mathscr{P}$ *and* $\mathscr{V}$
4: **end function**

4.2. The Evolutionary Computation Approach

Similar to the Iterated-KSP strategy, we used a Python library called `Inspyred` to develop the evolutionary computation approach (which will be called Iterated-EC in the rest

of the paper). `Inspyred` is a library of biologically-inspired algorithms including evolutionary computation, swarm intelligence, and neural networks. Often these terms are also known as computational intelligence.

The `Inspyred` library grew out of insights from [De Jong 2006]. The goal of the library is to separate problem-specific computation from algorithm-specific computation in a clean way so as to make algorithms as general as possible across a range of different problems. Any bio-inspired algorithm has at least two aspects that are entirely problem-specific: how solutions to the problem look like and how such solutions are evaluated. These components will certainly change from problem to problem [Garrett 2013].

Algorithm 2 Iterated-EC Strategy

Input: Number of physical hosts: $\mathscr{P}$
Input: Number of virtual machines: $\mathscr{V}$

1: **function** ITERATED-EC-SCENARIO(...)
2: ea ← inspyred.ec.EvolutionaryComputation
3: ea.selector ← inspyred.ec.selectors.tournament_selection
4: ea.variator ← list of inspyred.ec.variators.n_point_crossover, inspyred.ec.variators.bit_flip_mutation
5: ea.replacer ← inspyred.ec.replacers.generational_replacement
6: ea.terminator ← inspyred.ec.terminators.evaluation_termination
7: constraints ← build constraints for every d-dimension to be ≤ 100
8: placement ← maximize VMs using ea.`evolve` with constraints, $\mathscr{P}$, $\mathscr{V}$, $\mathscr{E}$ $\mathscr{G}$
9: **end function**

The Algorithm 2 is called from the Algorithm 3 on every host to determine the VMs to be placed within. Similar to the Iterated-KSP algorithm, it is assumed that another algorithm already built the physical hosts and VMs lists from a given trace workload. The difference from Iterated-KSP is that now we need to remap the VMs data into a structure compatible with `Inspyred`.

A list of constraints is built (line 7) for each d-dimension. The summation of each resource used by the VMs within a host must be less than 100%. As we stated in the beginning of this section there are algorithm-specific components (lines: 3, 4, 5 and 6), and problem-specific components (line 8), which are $\mathscr{E}$ (the evaluator) and $\mathscr{G}$ (the generator). Similar to the Iterated-KSP algorithm the objective is to maximize the number of VMs per physical host. This is done with the ea.`evolve` (line 8).

The generator function ($\mathscr{G}$) generates possible solutions where each bit corresponds to including/excluding the VM at that host, giving a 1% chance of each bit being one, since we want the vast majority of the candidates to be zeroes. The fitness evaluation function ($\mathscr{E}$) is measured by adding up the number of VMs d-dimensions and subtracting the total amount of constraints that goes over 100. Other generation and evaluation functions may be used. Further details can be found on Section 7.

5. The Simulation Framework and Experiment Methodology

The simulation framework is based on the Algorithm 3 that iterates over the available (unplaced) physical hosts scenarios $\langle PMS \rangle$ (line 2) and later over the VMs scenarios

$\langle VMS \rangle$ (line 3) to determine a placement using a given strategy $\mathcal{S}$ for that scenario of physical hosts and VMs. Depending on the strategy $\mathcal{S}$, Algorithm 1 or Algorithm 2 will be used on every host to determine the VMs to be placed within.

The placement will be performed by evaluating the next suitable physical host that can handle the workload of a given VM, if any. Iterated-KSP and Iterated-EC strategies are energy-aware so hosts that are underloaded are automatically suspended with the consequent energy savings, and reactivated when required using the Wake-on-LAN mechanism. This is the opposite case compared with the Energy-Unaware strategy that does not suspend any host.

Each VM resource usage is represented by a vector as we introduced in Section 4. On each simulation, the CPU is represented directly by the traces we analyzed. To represent the other resources (memory, network, I/O) we used the same traces but with one fourth shifting of the used trace.

Algorithm 3 Virtual Machines placement framework

Input: A Strategy: $\mathcal{S}$
Input: List of physical hosts scenarios: $\langle PMS \rangle$
Input: List of virtual machines scenarios: $\langle VMS \rangle$
Output: A placement solution

```
1: function SIMULATE-STRATEGY(...)
2:     for 𝒫 ∈ ⟨PMS⟩ do
3:         for 𝒱 ∈ ⟨VMS⟩ do
4:             Solution-scenario ← Simulate S with 𝒫 and 𝒱
5:         end for
6:     end for
7: end function
```

The algorithms strategies that can be used as input to Algorithm 3 are:

- Energy-Unaware: It is a simple reference strategy that when evaluating the i-th hosts sequentially, picks a random VM and if does not breaks the SLA then it is placed within that host, otherwise another VM is picked to check if again it is possible to place it until no more VMs or hosts are left.
- Iterated-KSP: As described on Algorithm 1
- Iterated-EC: As described on Algorithm 2

Consumed energy estimations are done linearly with the growth of the CPU utilization as proposed in [Fan et al. 2007] without loss of generality since our focus is to efficiently place the VMs regarding their resources use.

It is important to conduct experiments using workload traces from real systems rather than artificially generated ones, as this would help to reproduce a realistic scenario. So we analyzed more than 11,776 24-hour long traces provided as part of the CoMon project, a monitoring infrastructure for PlanetLab. 10 days of workload traces [2] collected during March and April 2011 have been randomly chosen as the data set. The traces

[2] `https://github.com/vonpupp/planetlab-workload-traces`

Proposed		Nearest			
Std	Mean	Std	Mean	No.	Trace filename
15	25	15.5204	25.0694	1	146-179_surfsnel_dsl_internl_net_root
15	35	15.9337	34.7465	2	host4-plb_loria_fr_uw_oneswarm
15	45	15.1083	44.7083	3	plgmu4_ite_gmu_edu_rnp_dcc_ufjf
30	25	30.1624	23.7673	4	planetlab1_fct_ualg_pt_root
30	35	30.4136	33.1076	5	host3-plb_loria_fr_inria_omftest
30	45	30.5568	46.5520	6	planetlab1_georgetown_edu_nus_proxaudio
45	25	35.0424	25.3125	7	planetlab1_dojima_wide_ad_jp_princeton_contdist
45	35	38.5349	35.4791	8	planetlab-20110409-filtered_planetlab1_s3_kth_se_sics_peerialism
45	45	43.5875	47.8680	9	planetlab-wifi-01_ipv6_lip6_fr_inria_omftest

Table 1. Trace scenarios with the selected traces

include data on the CPU utilization collected every 5 minutes from more than a thousand VMs deployed on servers located in more than 500 places around the world.

Later we analyzed each trace individually to obtain statistical indicators like minimum and maximum CPU usage, mean and standard deviation. We noticed that the overall standard deviation range was from 0.2634 to 43.5875, so we divided it into tree equal parts (15, 30, ≈45). The overall mean range was from 0.5173 to 95.9756. This time instead of dividing it into three equal parts we took $35 \pm 10\%$. The reason of using 35% as a base value is due to the observation in [Barroso and Holzle 2007], which states that the mean value of the CPU utilization is 36.44% with 95% confidence interval. The traces analysis is summarized on Table 1. Notice the fifth column (trace number), which is a trace identifier that we will be referring to on the rest of the paper.

The number of physical hosts is in the interval $[10, 100]$ with increments by 10. The number of VMs is in the interval $[16, 288]$ with increments by 16.

A simulation is made of a triplet of (trace, algorithm, hosts) with VMs varying on the interval $[16, 288]$.

6. Performance Evaluation

We repeated each simulation 30 times to check if there was a clear tendency, and later reduced the data to three cases:

- Best case: representing the best case among the 30 repetitions
- Worst case: representing the worst case among the 30 repetitions
- Average case: representing the average (mean) case among the 30 repetitions

The hardware used on the experiments was a VM with four dedicated cores and 10 Gigabytes of RAM hosted by an octo-core Intel(R) Core(TM) i7-2700K CPU @ 3.50GHz with 16 Gigabytes RAM host system.

The experiments resulted in more than 8,000 scenarios that have been later summarized into more than 1,900 graphs showing several aspects of each placement. We are going to present the most representative set of figures and our conclusions on the rest of this section. To allow the reproduction of our experiments, all the code, data, documentation and figures related to the experiments are publicly available at [3].

Figure 1 shows the average case of energy consumption for the three algorithms simulating the Trace 1 using 100 hosts, with VMs ranging from 16 to 288. Similarly,

[3] `https://github.com/vonpupp/sbrc-2014-simulation`

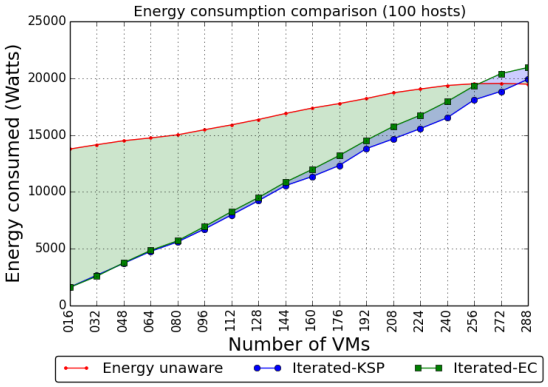

Figure 1. Energy consumption comparison - Trace 1 (100 hosts) - Average case

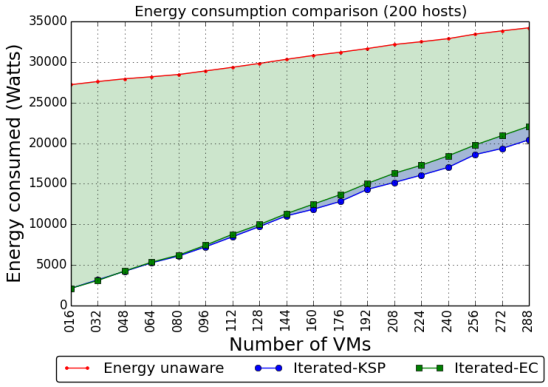

Figure 2. Energy consumption comparison - Trace 1 (200 hosts) - Average case

Figure 3 shows results of simulation of Trace 2 and Figure 4 shows results of simulation of Trace 3. The highlighted green area represents the energy savings from the Iterated-EC strategy (line with squares) in relation to the energy unaware strategy (line with dots), while the highlighted blue area represents the extra energy savings comparing the Iterated-EC strategy with the Iterated-KSP (line with circles). The lower the value, the better the strategy, hence the more energy savings. It is interesting to notice that near the intersection of the curves ($\sim$224 VMs on Figure 3 and $\sim$160 VMs on Figure 4) there is a saturation point where both strategies converge, this means that there are more VMs placed compared to the energy unaware strategy thus an optimal placement is achieved, this is also the reason why there is more energy consumption after the intersection of the curves compared with the energy unaware and therefore more VMs placed and a better use of the computation resources. After the saturation point the Iterated-EC tend to stabilize while the Iterated-KSP continues to optimize new VMs when there is no more resources for new VMs since resources are nearly exhausted.

As we can see there is a significant savings of energy with our proposed placement strategies, obviously the lower the overall cluster load (the less the VMs per physical host) the higher the energy savings and consequently the more physical suspended hosts. This can be observed by comparing Figures 1 and 2 which uses Trace 1 with 100 and 200 hosts respectively.

An observed tendency is that when the mean of resources usage increases (first three traces scenarios) a sooner saturation of VMs is achieved (fewer VMs can be placed). We also notice that when the standard deviation decreases the three algorithms tend to perform in a very similar way due to the fact that all VMs will have very similar resource needs.

Figure 5 shows the average case of unplaced VMs for the three algorithms simulating the Trace 2 using 100 hosts. The first strategy to fail to place a new VM is the energy unaware, even when this strategy places the VMs while there are resources available on the physical machines. As expected, it is less efficient than both Iterated-EC and

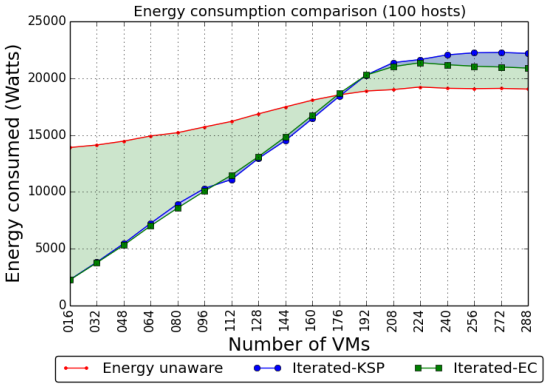

Figure 3. Energy consumption comparison - Trace 2 - Average case

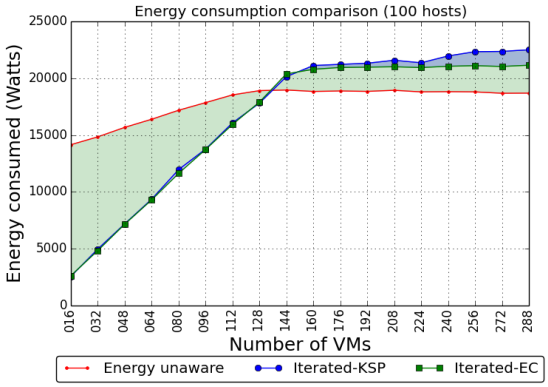

Figure 4. Energy consumption comparison - Trace 3 - Average case

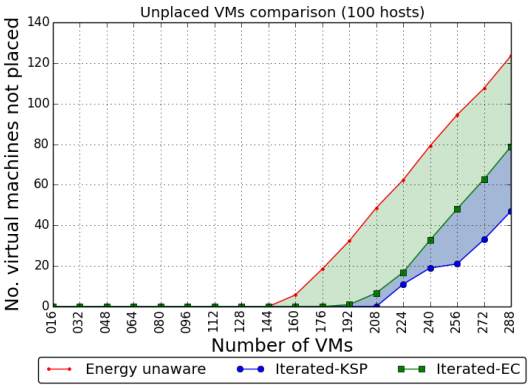

Figure 5. Unplaced VMs comparison - Trace 2 - Average case

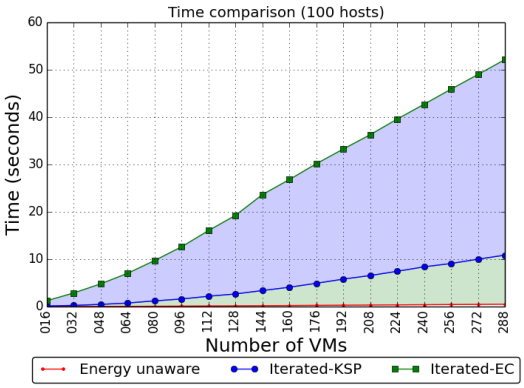

Figure 6. Exec. time comparison - Trace 3 - Average case

Iterated-KSP strategies. Iterated-KSP is slightly better than Iterated-EC by placing more VMs.

Figure 6 shows the execution time of the three strategies. The time taken by the Iterated-EC strategy is higher than the time taken by the Iterated-KSP strategy. This is due to the dependencies optimization of `OpenOpt` and our specific heuristic used on the Iterated-EC strategy.

Figure 7 shows the suspended physical machines for the three algorithms when simulating Trace 2 using 100 hosts. Both Iterated-EC and Iterated-KSP strategies are able to consume less energy due to the fact that they are able to place more VMs per physical host than the energy-unaware strategy.

We computed the 95% confidence intervals of the 30 experiments and we noticed that there is no significant difference. The energy unaware strategy is the only one who exhibits a broader confidence range due to the random behavior. The Iterated-EC strategy

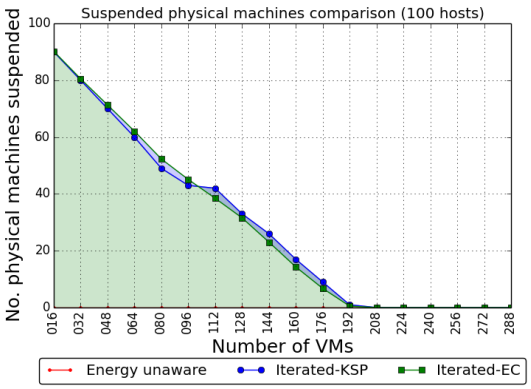

Figure 7. Suspended physical hosts comparison - Trace 2 - Average case

has insignificant variances and the Iterated-KSP strategy showed to be the most deterministic. Both showed predictable behavior.

7. Conclusion and Future Directions

As shown by the results, the proposed algorithms achieved significant energy savings. The Evolutionary Computation had energy savings starting from 35.46% for a workload of 288 VMs and up to 92.20% for a workload of 16 VMs with 200 hosts. The Knapsack based algorithm had energy savings starting from 40.33% for a workload of 288 VMs and up to 92.21% for a workload of 16 VMs. Concluding that the lower the overall cloud VMs workload the higher the energy savings compared to the energy unaware. We noticed that the Knapsack based approach is 7.55% better than the Evolutionary Computation approach (average case) which can be translated into a difference of 1667.70 Watts.

Both algorithms optimize hardware utilization on full placements compared to the energy unaware. The Knapsack based algorithm optimizes hardware by 6.20% to 20.40% however it is not stable. Evolutionary Computation ranges from 7.49% to 13.14% with a trend to be stable $\approx$ 11%. The Knapsack based algorithm is 11% to 15% faster than the Evolutionary Computation algorithm. The execution time difference tend to increase with the number of hosts and VMs at a rate of $\approx$ 5 seconds per 100 hosts.

There are other methods that are likely candidates to solve the problem of VMs placement that would be interesting to explore, among them we can cite: Greedy algorithms with different heuristics, Particle Swarm Optimization, Genetic Algorithms, Ant Colonies Optimization, Convex Hulls, Simulated Annealing, Tabu Search or Multicommodity flow. Following a *host-iterated* strategy should ease the integration process into our framework.

A specific approach has been proposed for the generation and evaluation functions on Algorithm 2, but there might be others that could be outperform the proposed ones. Also it would be possible to fine tune the parameters of the Iterated-EC algorithm to optimize the consumed time on computations. Another proposal is to run in parallel the Iterated-EC placement algorithm. This could be easily performed since the `inspyred`

installation process is straight forward.

Regarding the simulation framework, multithreading improvements can be done to the simulator so simulations can be delegated to different cores. It is important to only delegate full simulation scenarios which are thread-save, this is, the triplet described on Section 5.

It worth researching about making decisions on the distribution of resources among the physical hosts based on policies, i.e: it would be possible to have a physical servers pool for networking intensive VMs, another one for I/O intensive, etc. Another approach would be the opposite, try to distribute equally the resources d-dimensions across the physical hosts to optimize their use.

Acknowledgments

We would like to thank Aaron Garrett for his help and support using `Inspyred`. We also would like to thank Elaine Watanabe and Danilo Bellini for their positive feedback on how to improve this work.

References

[Barroso and Holzle 2007] Barroso, L. and Holzle, U. (2007). The case for energy-proportional computing. *Computer*, 40(12):33–37.

[Beloglazov and Buyya 2012] Beloglazov, A. and Buyya, R. (2012). OpenStack neat: A framework for dynamic consolidation of virtual machines in OpenStack clouds–A blueprint. Technical report, Technical Report CLOUDS-TR-2012-4, Cloud Computing and Distributed Systems Laboratory, The University of Melbourne.

[Beloglazov and Buyya 2013] Beloglazov, A. and Buyya, R. (2013). Managing overloaded hosts for dynamic consolidation of virtual machines in cloud data centers under quality of service constraints. *IEEE Transactions on Parallel and Distributed Systems*, 24(7):1366–1379.

[Beloglazov et al. 2010] Beloglazov, A., Buyya, R., Lee, Y. C., and Zomaya, A. (2010). A taxonomy and survey of energy-efficient data centers and cloud computing systems. arXiv e-print 1007.0066.

[Brian et al. 2008] Brian, H., Brunschwiler, T., Dill, H., Christ, H., Falsafi, B., Fischer, M., Grivas, S. G., Giovanoli, C., Gisi, R. E., and Gutmann, R. (2008). Cloud computing. *Communications of the ACM*, 51(7):9–11.

[Community project, supported by Parallels, Inc. 2013] Community project, supported by Parallels, Inc. (2013). OpenVZ linux containers wiki. Accessed: 2013-09-05.

[De Jong 2006] De Jong, K. (2006). *Evolutionary Computation: A Unified Approach*. Bradford Book. Mit Press.

[Dhiman et al. 2010] Dhiman, G., Mihic, K., and Rosing, T. (2010). A system for online power prediction in virtualized environments using gaussian mixture models. In *2010 47th ACM/IEEE Design Automation Conference (DAC)*, pages 807–812.

[Fan et al. 2007] Fan, X., Weber, W.-D., and Barroso, L. A. (2007). Power provisioning for a warehouse-sized computer. In *Proceedings of the 34th annual international symposium on Computer architecture*, ISCA '07, page 13–23, New York, NY, USA. ACM.

[Fettweis and Zimmermann 2008] Fettweis, G. and Zimmermann, E. (2008). ICT energy consumption-trends and challenges. In *Proceedings of the 11th International Symposium on Wireless Personal Multimedia Communications*, volume 2, page 6.

[Garrett 2013] Garrett, A. (2013). Inspyred inspired intelligence initiative. Accessed: 2013-11-13.

[Jackson and Lameter 2013] Jackson, P. and Lameter, C. (2013). CGROUPS kernel website.

[Kroshko 2013] Kroshko, D. L. (2013). OpenOpt website. Accessed: 2013-11-13.

[Linux Foundation Collaborative Project 2013] Linux Foundation Collaborative Project (2013). The xen project, the powerful open source industry standard for virtualization. Accessed: 2013-09-04.

[Linux-VServer Development Group 2013] Linux-VServer Development Group (2013). Linux-VServer. Accessed: 2013-09-05.

[LXC Development Group 2013] LXC Development Group (2013). LXC linux containers website. Accessed: 2013-09-02.

[Mell and Grance 2011] Mell, P. and Grance, T. (2011). The NIST definition of cloud computing (draft). *NIST special publication*, 800(145):7.

[Rajkumar Buyya 2013] Rajkumar Buyya, Rodrigo N. Calheiros, N. G. (2013). CloudSim website. Accessed: 2013-12-04.

[Redhat Emerging Technologies 2013] Redhat Emerging Technologies (2013). KVM (for kernel-based virtual machine) - main page. Accessed: 2013-09-04.

[Rich Brown 2007] Rich Brown (2007). Report to congress on server and data center energy efficiency:Public law 109-431.

[Soltesz et al. 2007] Soltesz, S., Pötzl, H., Fiuczynski, M. E., Bavier, A., and Peterson, L. (2007). Container-based operating system virtualization: a scalable, high-performance alternative to hypervisors. In *ACM SIGOPS Operating Systems Review*, volume 41, page 275–287.

[The Open Compute Project Foundation 2013] The Open Compute Project Foundation (2013). Open compute project, "Energy efficiency". Accessed: 2013-10-06.

[Vigliotti and Batista 2014] Vigliotti, A. D. L. F. and Batista, D. M. (2014). pyCloudSim Github repository. Accessed: 2014-03-21.

[VMware, Inc. 2013] VMware, Inc. (2013). VMware virtualization for desktop & server, application, public & hybrid clouds - united states. Accessed: 2013-09-04.

[Xavier et al. 2013] Xavier, M., Neves, M., Rossi, F., Ferreto, T., Lange, T., and De Rose, C. (2013). Performance evaluation of container-based virtualization for high performance computing environments. In *2013 21st Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, pages 233–240.